

HW#2 N-gram Language Models

트라이그램 문장 생성과 Unigram / Bigram / Trigram Perplexity 비교

과목	LLM 프로그래밍 (Lab2_Ngram)
제출자	이름 _____ 학번 _____
저장소	https://github.com/lyusung-assignment/LLM-prog-env
소스	llm-docker/workspace/hw2/hw2_ngram.py

1. 구현 개요

슬라이드의 restaurant 코퍼스 15문장으로 unigram · bigram · trigram 모델을 하나의 NgramLM 클래스로 구현했다. 차수만 바꾸면 같은 코드가 세 모델을 모두 처리한다. 표준 라이브러리만 사용한다.

- **add-k 스무딩**. MLE만 쓰면 학습에서 못 본 n-gram 하나 때문에 문장 확률이 0, perplexity가 inf가 되어 세 모델을 비교할 수 없다. 따라서 $P(w|ctx) = (C(ctx, w) + k) / (C(ctx) + kV)$ 를 적용했다 (기본 $k=1$).
- **비교의 공정성**. n차 모델은 $\langle s \rangle$ 를 n-1개 붙여 패딩한다. 그래야 세 모델이 테스트 문장의 **같은 토큰**($w_1 \dots w_m, \langle /s \rangle$)을 예측하게 되고, 분모 N이 같아져 perplexity를 그대로 비교할 수 있다.
- **OOV 처리**. 테스트 문장에만 나오는 단어는 $\langle unk \rangle$ 로 매핑하고 어휘 V에 한 칸을 할당한다.
- **생성은 MLE 분포로**. 스무딩된 분포로 샘플링하면 어휘 전체에 확률이 조금씩 퍼져 문맥과 무관한 단어가 섞인다. 생성은 원래 카운트로 하고, 못 본 2-word 컨텍스트는 bigram $\rightarrow$ unigram 순으로 backoff 한다.

2. 문제 1 — 트라이그램 모델로 문장 5개 생성

트라이그램 확률 $P(w_i | w_{i-2}, w_{i-1}) = C(w_{i-2}, w_{i-1}, w_i) / C(w_{i-2}, w_{i-1})$ 로 분포를 만들고, $\langle s \rangle \langle s \rangle$ 에서 시작해 $\langle /s \rangle$ 가 나올 때까지 샘플링한다. 난수 seed는 42로 고정했다.

#	생성된 문장	비고
1	can you find a thai restaurant	훈련 문장과 동일
2	i want chinese food	훈련 문장과 동일
3	i want to eat	훈련 문장과 동일
4	i am looking for a restaurant	훈련 문장과 동일
5	where can i eat chinese food	훈련 문장과 동일

생성된 5문장이 모두 훈련 문장과 일치한다. 훈련 문장이 15개뿐이라 두 단어 컨텍스트의 다음 단어가 거의 유일하게 결정되기 때문이다 (예: $P(\text{to} | \text{i}, \text{want}) = 3/7$ 을 빼면 나머지 컨텍스트는 대부분 후보가 1개다). 트라이그램 모델이 코퍼스를 사실상 암기한 상태이며, 이것이 문제 2에서 측정할 sparsity 문제로 이어진다.

3. 문제 2 — Perplexity 비교

훈련에 쓰지 않은 문장 8개로 측정했다 (과제 요구사항은 5개 이상). 코드에서 `assert`로 훈련 문장과 겹치지 않음을 검증한다. 마지막 문장에는 미등록 단어 `korean`을 일부러 넣어 OOV 처리를 확인했다.

$$PP(W) = \exp\left(-\frac{1}{N} \cdot \sum \log P(w_i \mid \text{context})\right), \quad \text{add-k smoothing, } k = 1.0$$

테스트 문장 (훈련에 없음)	unigram	bigram	trigram
i want to eat italian food	17.19	7.41	9.47
i want a chinese restaurant	14.33	7.99	9.96
can you find italian food	20.91	9.25	12.38
tell me about italian restaurants	29.51	13.37	15.64
i am looking for a thai restaurant	21.71	8.94	10.63
where can i eat thai food	18.87	11.66	13.96
i am looking for italian food	22.17	8.88	11.41
where can i find a korean restaurant	25.27	15.53	20.17
전체 corpus perplexity	20.89	10.13	12.65
(참고) 훈련 문장에 대한 perplexity	18.31	7.73	8.74

해석

- **unigram이 가장 나쁘다.** 컨텍스트를 전혀 쓰지 않으므로 어떤 단어가 앞에 오든 같은 분포를 쓴다. 단어마다 거의 어휘 크기만큼의 불확실성을 그대로 떠안는다.
- **bigram이 가장 좋다 (PP = 10.13).** 한 단어 컨텍스트만으로도 후보가 크게 줄고, 필요한 bigram(want to, italian food 등)은 훈련에서 이미 관측됐다.
- **trigram은 훈련 문장에서는 bigram보다 낫지만(8.74 vs 7.73) 테스트 문장에서는 더 나쁘다(12.65 vs 10.13).** 훈련 데이터에 과적합된 것이다. 못 본 2-word 컨텍스트는 카운트가 0이라 add-k의 균등분포 $1/V$ 로 떨어지는데, 이 비용이 bigram의 더 날카로운 추정정보다 크다.
- 즉 컨텍스트를 늘리는 것은 **데이터가 충분할 때만** 이득이다. 15문장 규모에서는 트라이그램의 표현력보다 sparsity의 손해가 크다.

스무딩 상수 k에 따른 변화

위 순위가 $k=1$ 이라는 선택 때문에 생긴 것은 아닌지 확인했다.

k	unigram	bigram	trigram
1	20.89	10.13	12.65
0.1	21.89	4.75	6.31
0.01	22.87	4.39	5.58
0.001	23.86	5.58	7.46

k를 0.001까지 낮춰도 **bigram < trigram** 순서는 유지된다. 이 결과는 스무딩 상수가 아니라 코퍼스 크기에서 오는 성질이다.

4. 실행 결과

```
$ python hw2_ngram.py
```

```
=====
Problem 1. Trigram model - sentence generation
=====
training sentences : 15
vocabulary size V : 24
distinct trigrams : 56

P(w | i, want):
  to      3/7 = 0.429
  chinese 1/7 = 0.143
  thai    1/7 = 0.143
  italian 1/7 = 0.143
  a       1/7 = 0.143

5 generated sentences:
1. can you find a thai restaurant      (seen in training)
2. i want chinese food                 (seen in training)
3. i want to eat                      (seen in training)
4. i am looking for a restaurant      (seen in training)
5. where can i eat chinese food        (seen in training)

5/5 are copies of a training sentence. With 15 sentences
a two-word context almost always has one continuation, so the model
mostly replays what it saw.

=====
Problem 2. Perplexity on held-out sentences
=====
test sentences: 8, none used in training
smoothing      : add-k, k = 1.0

test sentence                unigram    bigram    trigram
-----
i want to eat italian food    17.19     7.41     9.47
i want a chinese restaurant   14.33     7.99     9.96
can you find italian food     20.91     9.25    12.38
tell me about italian restaurants 29.51    13.37    15.64
i am looking for a thai restaurant 21.71     8.94    10.63
where can i eat thai food     18.87    11.66    13.96
i am looking for italian food  22.17     8.88    11.41
where can i find a korean restaurant 25.27    15.53    20.17
-----
corpus perplexity            20.89     10.13    12.65
(on the training sentences)   18.31     7.73     8.74

Best on the test set: bigram, PP = 10.13
Unigram ignores the context entirely, so it is the worst. Trigram wins
on the training sentences but loses on the test set: its contexts are
too sparse, and an unseen context falls back to the flat 1/V estimate.

=====
Test perplexity vs. the smoothing constant k
=====
      k      unigram      bigram      trigram
-----
      1      20.89      10.13      12.65
     0.1      21.89       4.75       6.31
    0.01      22.87       4.39       5.58
   0.001      23.86       5.58       7.46

Bigram stays ahead of trigram at every k, so the ranking comes from
the size of the corpus, not from the smoothing constant.
```